

Notes from Recall

Lecture II: Intro to LL & OOP

→ There are a couple of programming paradigms present

1) Procedural → function based programming, where fⁿs orchestrate the entire control flow of the application.

→ in short, bunch of fn's calling each other
languages → C etc.

2) OOP → earlier action was being performed on object
→ here, object performs an action

function `getDetails(String name, int age) { ... }`

procedural: `getDetails("John", 24) ...`

OOP: `student.getDetails()`:



object is performing an action
providing more control & structure

Pillars of OOP

1) Abstraction → exposing just enough information to get the work done.

→ informatⁿ on internal details is not a necessity to perform the action

Arrays.sort ()

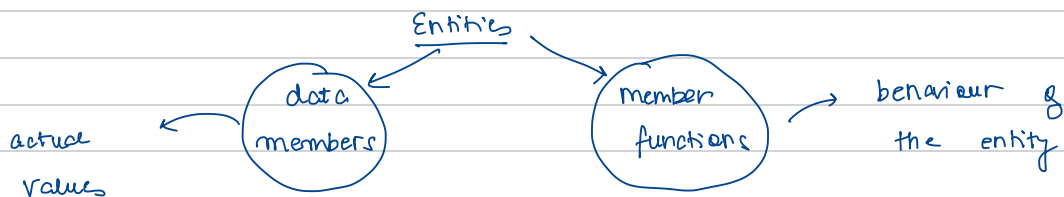
↳ we can use it for sorting w/o needing internal info

comparator

2) Encapsulation



In OOP, we deal with entities.



We want to

- 1) Bind & hold the data members + functions together
- 2) protect them from the outside world, just like a capsule

↳ The act of achieving this principle is called Encapsulation & we achieve it through the use of classes in Java

```
class Student {
```

```
    String name;  
    private int rollNo;
```

```
    void printName () {};
```

```
    public getRollNo () {};
```

```
}
```

{ } → bind together

Access Modifier

→ protect / controlled access to outside world

Access Modifiers

private : only within class

default : only within same package

protected : only within same package + outside package, child class

public : anyone anywhere can access

→ class is only a blueprint.

→ This blueprint is used to create actual objects which hold actual data & physical memory.

↳ Student student = new Student ();

↓

↳ memory allocated

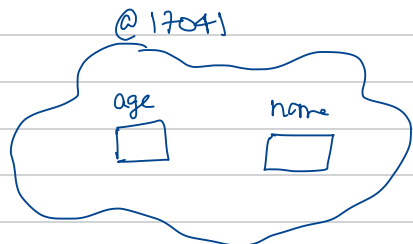
↳ constructor

→ since objects hold data in their own custom way,
∴ classes are called as custom data types

→ new keyword allocates memory, whereas the constructor fn defines how & what all values will the data be initialized with.

→ object variables are reference variable and they only store the physical address of where the object is created in memory.

Student
@17041



→ If we don't provide a constructor for a class, the java compiler provides a no-args default constructor.

→ The moment we define our own constructor, the default constructor is no longer provided and the complete control & responsibility for initialisation is handed over to the class.